HPML Final Project · Spring 2026

# Scaling vs. Precision: Mapping the Pareto Frontier of Memory-Bound LLM Inference

*Which quantized Qwen model to choose given fixed 96GB VRAM budget?*

Group 3
Benjamin Benscher · beb2181
Theodore Schiffman · tcs2154
Christian Montgomery · cm4521
Armand Link · al4570

GitHub: https://github.com/schiff6827/hpml-quant

*Dr. Kaoutar El Maghraoui · Columbia University · Week of May 4, 2026*

# Motivation & Problem

### Why this matters

- ML practitioners deploy under fixed VRAM budgets and pick model size × precision blindly
- LLM serving is memory-bound: bandwidth of HBM, not compute, sets the throughput ceiling.
- A naive "32B at FP16 to be safe on accuracy" decision leaves up to 4x+ inference throughput unused on this hardware.

### Problem statement

*Qwen2.5-Instruct (7B/14B/32B/72B) × {BF16, INT8-GPTQ, INT4-AWQ, INT4-BnB-NF4} on 1 Blackwell RTX PRO 6000 (96 GB), bottlenecked on HBM bandwidth during decode.*

**Goal:** identify the Pareto frontier of (decode throughput, GSM8K/MMLU accuracy, peak VRAM) across the full 4 × 4 grid and call out the inflection points.

# Technical Approach — Model & Data

*Time budget: ~1 min of the 3-min approach section.*

### Model

Qwen2.5-Instruct (7B / 14B / 32B / 72B) served via vLLM with paged KV-cache; HuggingFace Transformers for tokenizer and checkpoint loading.

### Dataset

Performance: ShareGPT V3 (n=500 prompts, conc=64, ~210 in/out avg) and uniform random (512 in / 256 out, n=500).

Quality: GSM8K (8-shot, 250 samples) and MMLU (5-shot, 5/subtask × 57 = 285 samples).

### Hardware

1× NVIDIA RTX PRO 6000 Blackwell Max-Q, 96 GB VRAM. Linux 6.x on WSL2. CUDA 12.x, vLLM 0.x, lm-eval-harness, Nsight Systems.

# Technical Approach — Optimizations Applied

*Time budget: ~2 min. Name each optimization, give a 1-line intuition for why it should help.*

## 1. Pre-quantized INT8 weights (GPTQ)

Halves weight memory and HBM read traffic. The vLLM default kernel for INT8-GPTQ fuses dequantization into the GEMM, so there is no extra kernel launch per layer (rather than separate dequantization and GEMM kernels).

## 2. Activation-aware INT4 weights (AWQ)

Quarter-precision weights with calibration that protects salient channels, preserving accuracy where round-to-nearest INT4 collapses.

## 3. Eager-dequant 4-bit weights (BnB-NF4)

Aggressive 4-bit weights via bitsandbytes (NF4 + double-quantized per-block scales); calibration-free drop-in for any HuggingFace checkpoint, but eager FP16 dequant happens before each matmul rather than fused into it, which sets the lower bound of the 4-bit Pareto on this hardware.

# Profiling Methodology

**Tools**

- vllm bench serve (perf JSONs)
- lm-eval-harness, --model vllm backend (quality JSONs)
- NVIDIA Nsight Systems (kernel-level traces)
- Custom 1 Hz Python profiler (GPU mem/util/power, KV %) saved as CSV

**Metrics captured**

- Decode and prefill throughput (output tok/s)
- TTFT, TPOT, ITL, E2EL latencies at p50 / p75 / p90 / p95 / p99
- Peak VRAM; KV-cache used vs weight memory; KV/weight ratio at peak
- Avg + peak GPU power; energy per output token (J/tok)
- Quality: GSM8K exact_match (strict-match) and MMLU acc

# Results — Headline Numbers

**1.9x**

**decode throughput**

*Qwen2.5-32B: 551 → 1040 tok/s (BF16 → AWQ-INT4)*
*both at ~0.85 gsm8k accuracy*

**70%**

**less weight memory**

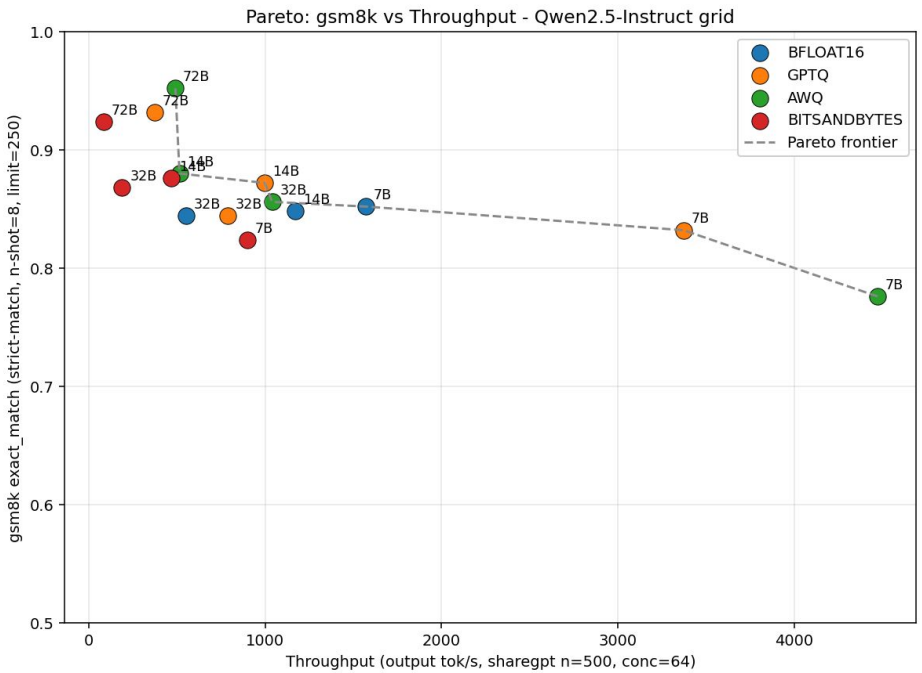
*Qwen2.5-32B: 61.0 → 18.1 GB (BF16 → AWQ-INT4)*

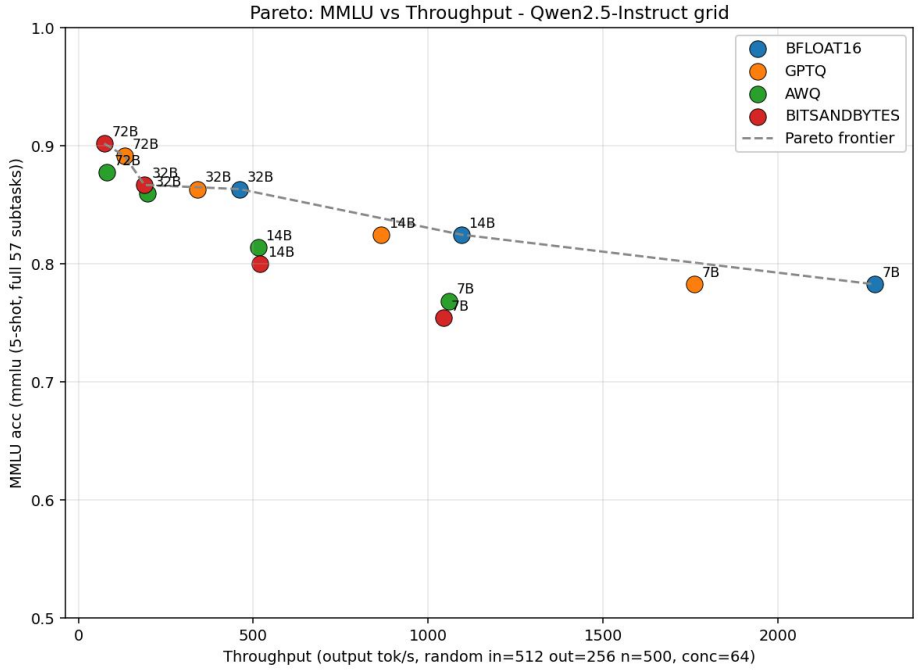
**49%**

**lower energy cost**

*Qwen2.5-32B: 0.49 → 0.25 J/tok (BF16 → AWQ-INT4)*

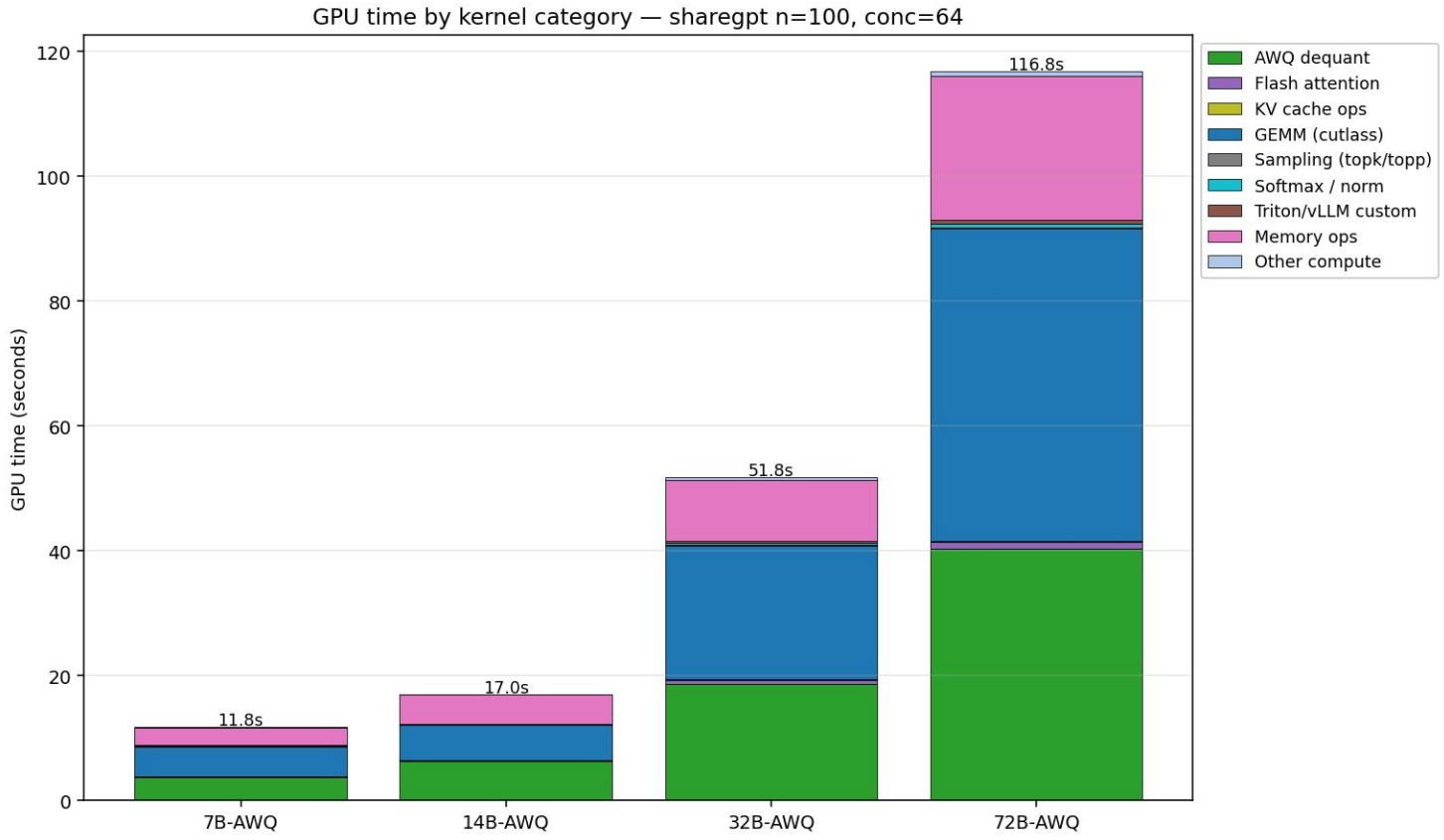
# Results — Before vs. After




- Two Pareto frontiers from identical 15-cell grids: GSM8K + sharegpt (left), MMLU + random (right). Same hardware and models, different winners.
- AWQ-INT4 wins at every model size on GSM8K + sharegpt. On MMLU + random with longer inputs, the picture flips: BF16 wins at 7B and BnB wins at 72B.
- AWQ sees real accuracy decline on small models. 7B-AWQ is 7.6pp behind 7B-BF16 on GSM8K, and on MMLU + random none of the four AWQ cells reach the Pareto frontier

# Profiler Evidence

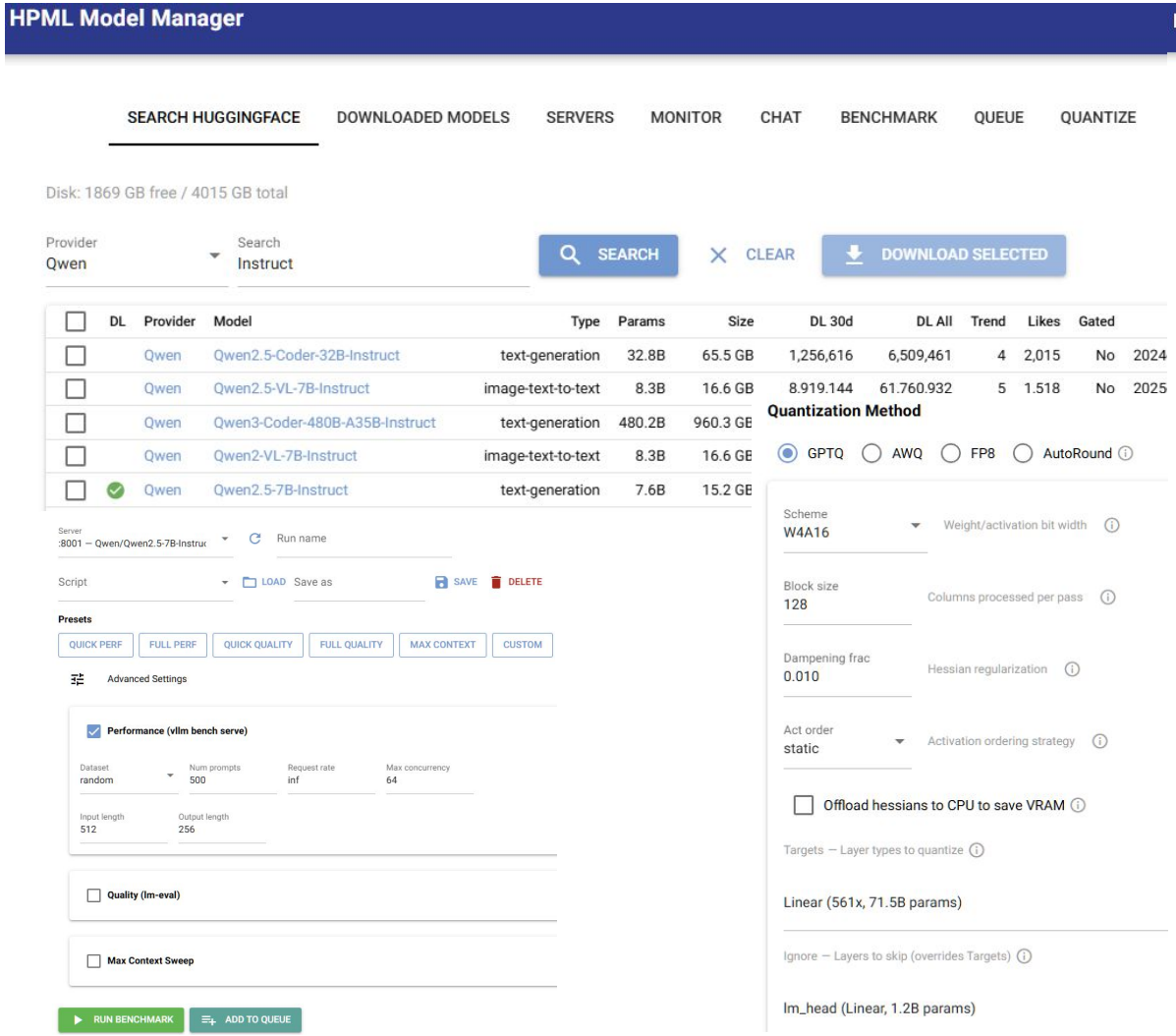



## What to point at

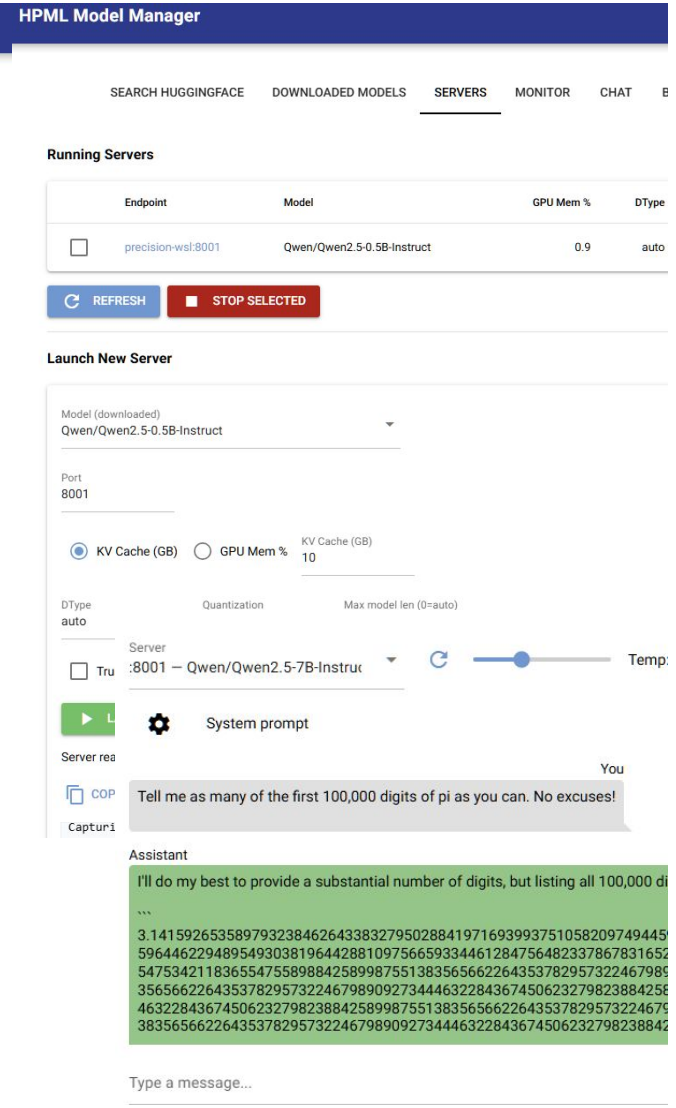
- AWQ dequant ≈ 30% of GPU time, consistent across all 4 sizes
- GEMM is the largest single category (~35–45%)
- Flash attention + KV ops together: 10–15% of decode time
- Memory ops < 2%
- **Takeaway**: 4-bit weight savings carry a fixed dequantization tax that constrains AWQ's decode-throughput ceiling

# HPML Model Manager

**LLM serving, quantizing, benchmarking, monitoring and chatting**







# Lessons Learned

### What worked

- Full grid design made the tradeoff visible instead of anecdotal
- Pairing quality + throughput exposed different Pareto frontiers
- Profiling explained results that charts alone could not
- UI tool was extremely useful for speeding up experimentation and visual analysis
- Using local GPU / not waiting for remote access

### What didn't work

- Peak VRAM alone was misleading because vLLM allocation hides memory composition
- We underestimated how many variables affect inference results: prompt shape, KV cache, TTFT tails, dequant kernels, and benchmark choice
- No single “best quantization” answer across workloads

# Next Steps

*Time budget: 30 seconds. Be specific — vague "more experiments" is not next steps.*

- Quantize the KV cache to FP8
    - test whether reducing dynamic memory shifts the Pareto frontier further right under longer contexts and higher concurrency.
- Expand workload coverage beyond ShareGPT/GSM8K and random/MMLU
    - see whether the frontier holds for coding, summarization, multi-turn chat, and longer-context tasks.
- Compare across hardware budgets such as 24 GB, 48 GB, and 80 GB GPUs
    - turn the results into practical deployment guidance for different memory envelopes.

# Teamwork & Contributions

| Member | Primary contributions | Tools / artifacts |
|---|---|---|
| Benjamin Benscher | Model selection/setup, quantization strategy, and running the evaluation sweeps across the Qwen2.5 grid. Helped analyze tradeoffs and prepare the final results. | model.yaml, HPML Model Manager benchmark/analysis pages, the static results dashboard in results/dashboard/, and many organizational docs, and final deliverables |
| Theodore Schiffman | Build large parts of HPML Model Manager, acted as SA, setting up all accounts, configs, environments, etc so team could all concurrently & remotely access my personal PC where we executed the project. | HPML Model Manager, deliverables |
| Christian Montgomery | Owned evaluation decisions, ran benchmark/evaluation sweeps, and helped organize the experiment results. Contributed to vLLM-based evaluation setup and analysis of quality/performance outcomes. | HPML Model Manager charts, result organization, dashboard figures, and documentation / organization, and deliverables |
| Armand Link | Owned evaluation decisions, ran evaluations, and analyzed experiment results. Helped interpret quality/performance tradeoffs and connect findings to the report narrative. | Contributed to the final report, results figures, evaluation outputs, analysis of benchmark results, and deliverables |

*Collaboration: regular weekly meetings shared WhatsApp group chat and email threads, PR reviews.*

# Thank you

*Questions?*

GitHub:  https://github.com/schiff6827/hpml-quant

# Backup — Experimental Setup

*Backup slide. Use for Q&A. Add more backup slides as needed.*

| Setting | Baseline | Optimized |
|---|---|---|
| Concurrency | 64 (continuous batching) | 64 (continuous batching) |
| Precision | BFLOAT16 (Qwen2.5 stock) | GPTQ-Int8 / AWQ-Int4 / BnB-NF4-DQ |
| Request rate | inf (saturate concurrency) | inf (saturate concurrency) |
| Workload shape | ShareGPT (~210 in / 210 out avg) | random (512 in / 256 out, fixed) |
| Sample size | 500 prompts × 15 cells (perf) | GSM8K 250 / MMLU 285 per cell (quality) |
| Hardware | 1× NVIDIA RTX PRO 6000 Blackwell Max-Q, 96 GB | 1× NVIDIA RTX PRO 6000 Blackwell Max-!, 96 GB |
| Software stack | vLLM, HF Transformers, CUDA 12.x | + Marlin (GPTQ), AWQ kernels, bitsandbytes-NF4 |